

FIAP SOAT — Tech Challenge Fase 4

Sistema de Ordens de Serviço da Oficina Mecânica — Arquitetura de Microserviços

1. Participantes

Nome	RM	E-mail
(preencher)	(RM)	(e-mail)
(preencher)	(RM)	(e-mail)

2. Links dos repositórios

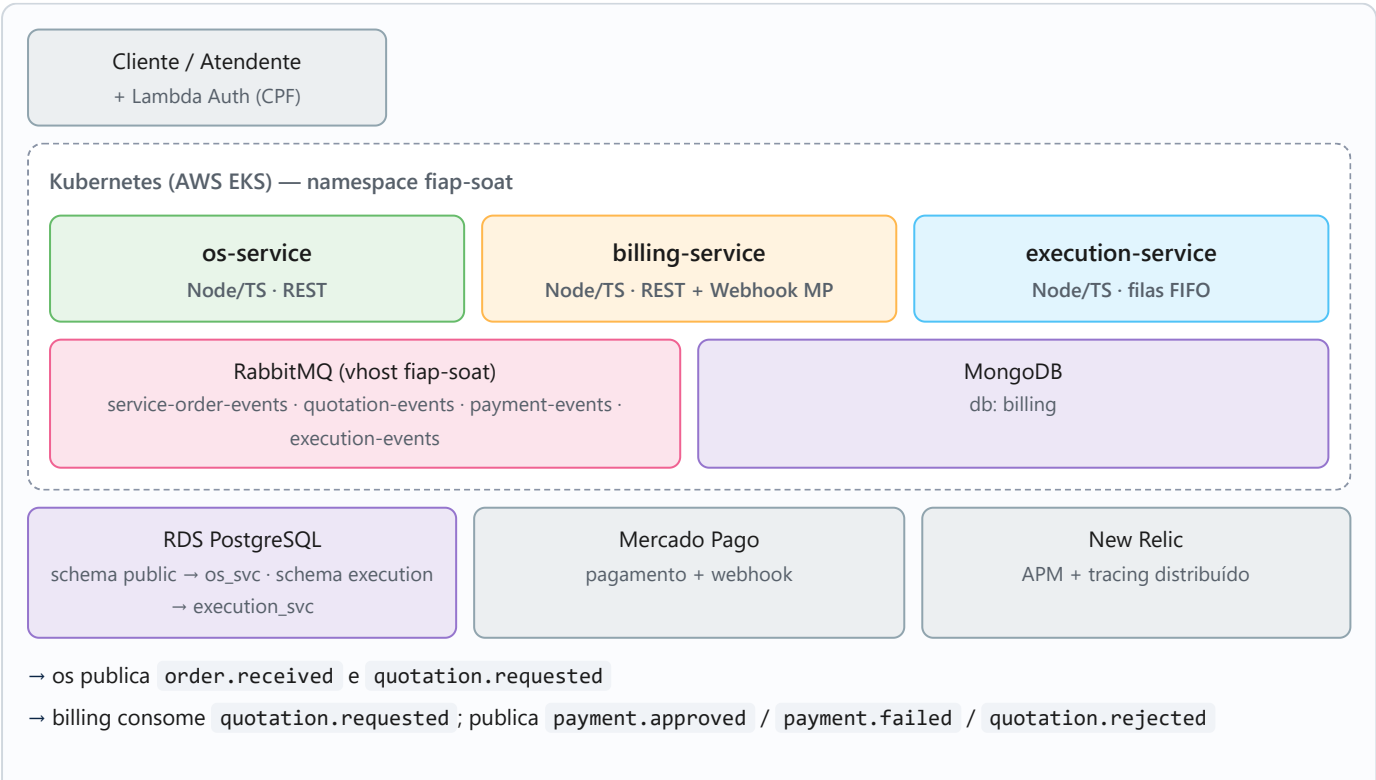
Microserviço	Repositório
OS Service	https://github.com/zmathmatos/fiap-soat-os-service
Billing Service	https://github.com/zmathmatos/fiap-soat-billing-service
Execution Service	https://github.com/zmathmatos/fiap-soat-execution-service
Infraestrutura (EKS + RDS + RabbitMQ + Mongo)	https://github.com/zmathmatos/fiap-soat-tech-challenge-infra-db
Lambda de autenticação (CPF)	https://github.com/zmathmatos/fiap-soat-tech-challenge-lambda

3. Vídeo de demonstração

Link (YouTube/Vimeo, ≤15 min, público ou não listado): (preencher)

Demonstra: fluxo completo de uma OS pelos 3 microserviços, execução do Saga Pattern e compensação em falha, deploy automatizado com validação de testes, e monitoramento/rastreamento distribuído (New Relic).

4. Diagrama geral da arquitetura



→ execution consome `order.received` e `payment.approved` ; publica `diagnostic.finished` / `execution.finished` / `execution.failed`

→ os consome `payment.*` e `execution.*` para atualizar o status da OS

Serviço	Banco	Isolamento
os-service	PostgreSQL (RDS) — schema <code>public</code>	role <code>os_svc</code> (GRANT só neste schema)
execution-service	PostgreSQL (RDS) — schema <code>execution</code>	role <code>execution_svc</code> (GRANT só neste schema)
billing-service	MongoDB — database <code>billing</code>	usuário <code>billing</code> com <code>readWrite</code> só neste db

Nenhum serviço acessa o banco de outro — imposto por permissões do PostgreSQL (`REVOKE ... FROM PUBLIC` + `GRANT` restrito por schema/role) e por usuário dedicado no MongoDB, não por convenção. Requisito SQL + NoSQL atendido (PostgreSQL + MongoDB).

5. Estratégia do Saga Pattern: Coreografia

Escolhemos **saga coreografada** (choreography): não há orquestrador central. Cada serviço **publica os eventos das etapas que executa** e **reage** aos eventos dos outros para dar continuidade ao fluxo. Cada *topic exchange* pertence ao seu publicador.

Fluxo feliz

1. Abertura da OS (os) → `order.received` → execution enfileira para diagnóstico.
2. Diagnóstico registrado (execution) → `diagnostic.finished` → os move a OS para "Aguardando aprovação" → `quotation.requested`.
3. billing consome `quotation.requested` → gera orçamento e envia por e-mail.
4. Cliente aprova → billing cria preferência no Mercado Pago; pagamento confirmado (webhook) → `payment.approved` → os move a OS para "Em execução" e execution inicia o reparo.
5. Reparo finalizado (execution) → `execution.finished` → os conclui a OS ("Finalizado").

Rollback / compensação

- **Orçamento rejeitado** → `quotation.rejected` → os/execution encerram/cancelam a OS.
- **Pagamento recusado/cancelado** → `payment.failed` → os move a OS para "Finalizado" e execution cancela a execução.
- Entrega **atômica** via *outbox* (estado + evento na mesma transação) e **idempotência** por `messageId` / `serviceOrderId` nos consumidores (at-least-once sem duplicar efeitos).

Por que coreografia: fluxo curto e de baixo acoplamento; evita ponto único de coordenação/falha, mantém cada serviço autônomo e escala melhor no cenário de múltiplas filiais. A menor visibilidade do fluxo distribuído é mitigada pelo **tracing distribuído no New Relic**.

6. Justificativa da divisão dos microsserviços e tecnologias

Divisão por bounded context (DDD)

- **os-service** — ciclo de vida da OS e cadastros (usuários, veículos, peças, serviços). Dono do status da OS.
- **billing-service** — orçamento e pagamento (Mercado Pago). Isolado por lidar com dinheiro e provedor externo, com dados documentais (payload de pagamento variável → NoSQL).
- **execution-service** — filas FIFO de diagnóstico e execução do reparo. Isolado pela regra de ordenação estrita e cadência própria da oficina.

Área	Escolha	Motivo
Runtime	Node.js 22 + TypeScript	Produtividade, tipagem, ecossistema
Arquitetura	Clean Architecture (4 camadas)	Testabilidade; dependências apontando ao domínio
SQL	PostgreSQL (os, execution)	Dados relacionais/transacionais (OS, filas)
NoSQL	MongoDB (billing)	Payload de pagamento flexível/semiestruturado
Mensageria	RabbitMQ (topic exchanges)	Integração assíncrona desacoplada da saga
Pagamento	Mercado Pago	Requisito do desafio
Orquestração	Kubernetes (AWS EKS) + HPA	Deploy automatizado e escala horizontal
CI/CD	GitHub Actions → ECR → EKS	Pipeline independente por serviço
Qualidade	SonarCloud + cobertura ≥80%	Verificação automática no CI
Observabilidade	New Relic (APM + tracing)	Rastreamento dos fluxos distribuídos

7. Testes e qualidade

- **Testes unitários** em todos os serviços; **cobertura mínima de 80%** enforced no CI (Jest + SonarCloud).
- **BDD (Cucumber)** cobrindo fluxo completo + compensação da saga: `os-service ( features/os-flow.feature )`, `billing-service ( features/billing-flow.feature )`, `execution-service ( features/execution-flow.feature )`.
- **SonarCloud** por repositório (Quality Gate + cobertura) — badges nos READMEs.
- **CI/CD independente** por serviço (build → testes → Sonar → deploy EKS) e **branch main protegida** com PR obrigatório e checagens.

8. Nota de arquitetura — banco único com isolamento lógico

Por limite de crédito do AWS Academy, os serviços SQL compartilham uma única instância RDS com isolamento por schema + role (em vez de uma instância por serviço). O isolamento de acesso exigido pelo desafio está garantido por permissões do PostgreSQL. A migração para instâncias dedicadas não exige mudança de código — apenas de connection string. Detalhes no README do `fiap-soat-tech-challenge-infra-db`.